

Drive Aura 51

Manuale di installazione e verifica offline

Secondo progetto di Programmazione Web - scelta B - Versione candidata: 1 settembre 2026

Questo pacchetto migra i dati del Servizio Sanitario dal servizio PHP remoto a PostgreSQL locale. Il percorso applicativo è:

PHP remoto -> servlet Java/Tomcat -> Django -> PostgreSQL.

Il controllo rapido offline usa una piccola sorgente contrattuale in locale. Attraversa la servlet, Django e PostgreSQL reali, ma non sostituisce la prova massiva PHP/PDO già eseguita.

1. Prerequisiti

Usare Windows e il software già installato:

- 1 Python 3.12 x64 completo di `venv`;
- 2 Java 8 o successivo per Tomcat 9;
- 3 Java 17 o successivo per Tomcat 11;
- 4 Tomcat 9 oppure Tomcat 11, non Tomcat 10;
- 5 PostgreSQL da 14 a 18, avviato e raggiungibile.

Non servono Internet, Maven, Composer, Node.js, IDE o compilazione Java. Windows PowerShell 5.1 è normalmente incluso in Windows: il file `verifica-rapida.bat` lo avvia automaticamente. L'utente PostgreSQL deve poter creare database nuovi. Tenere a portata di mano i percorsi completi di `python.exe`, della directory JDK, della directory Tomcat e della cartella `bin` di PostgreSQL.

Non è obbligatorio aggiungere questi programmi al `PATH`: il BAT e il configuratore accettano i percorsi completi. Se invece si usa il `PATH`, controllare `python --version`, `java -version` e `psql --version`. Se un comando non è riconosciuto, indicare il percorso completo richiesto dal BAT o da `config\parameters.example.ps1`, senza modificare il sistema.

Controllare le versioni:

```
& 'C:\Python312\python.exe' --version
& 'C:\Program Files\Java\jdk-17\bin\java.exe' -version
& 'C:\apache-tomcat-11\bin\version.bat'
& 'C:\Program Files\PostgreSQL\18\bin\psql.exe' --version
```

Versioni supportate: Python **3.12 x64** completo di `venv` (non embeddable), PostgreSQL **14-18**, Tomcat **9 o 11**. Tomcat 10 non è supportato. Abbinare Tomcat 9 a Java 8+ e Tomcat 11 a Java 17+; il configuratore seleziona il WAR compatibile. Le versioni diverse da queste non sono garantite: installare una versione supportata invece di forzare l'avvio.

2. Estrarre e controllare il pacchetto

Importante: estrarre direttamente in `C:\DriveAura51`. PowerShell 5.1 può fallire durante `Expand-Archive` se la directory padre è molto lunga; nessuno script può prevenire un errore già avvenuto in questa fase.

- 1 Mettere ZIP e file `.sha256` nella stessa cartella.
- 2 Confrontare il checksum esterno prima di estrarre lo ZIP.
- 3 Estrarre lo ZIP nella cartella nuova `C:\DriveAura51`.
- 4 Aprire `drive-aura-51-offline` e avviare `verifica-rapida.bat`.

```
$expected = (Get-Content '.\drive-aura-51-offline.zip.sha256' -Raw).Split()[0]
$actual = (Get-FileHash '.\drive-aura-51-offline.zip' -Algorithm SHA256).Hash.ToLower()
if ($actual -ne $expected) { throw 'Checksum ZIP non valido' }
Set-ExecutionPolicy -Scope Process Bypass
.\installer\Test-PackageIntegrity.ps1
```

Il comando sopra è l'alternativa PowerShell diretta. Il BAT esegue lo stesso controllo interno automaticamente prima di chiedere qualsiasi dato. Attendere **Integrità pacchetto valida**. Se il controllo fallisce, eliminare la copia estratta e ripartire dallo ZIP originale. Il checksum non è un compito aggiuntivo: protegge da allegati incompleti o alterati. Il primo script controlla anche il budget dei percorsi prima di traversare o copiare file. Se è insufficiente, chiede di riestrarre in `C:\DriveAura51` e si ferma senza spostare o cancellare nulla.

3. Verifica rapida consigliata da cmd

Fare doppio clic su `verifica-rapida.bat`, oppure eseguire da Prompt dei comandi:

```
verifica-rapida.bat
```

Il BAT chiede i percorsi completi, utente/password del **PostgreSQL locale** e due nomi di database nuovi. Genera quattro segreti soltanto nella memoria della sessione per la sorgente sintetica e la verifica locale; non li salva e non richiede rete o token Altervista.

Se il database operativo o quello di verifica esiste già, il BAT si ferma e chiede di scegliere un nome nuovo: non cancella, svuota o sovrascrive database esistenti.

4. Alternativa PowerShell diretta

Usare questa alternativa solo se si preferisce configurare manualmente i parametri. Impostare i valori soltanto nella sessione PowerShell. Non modificare i file del pacchetto e non passare segreti come parametri ai processi. Disabilitare prima il salvataggio della cronologia della sessione:

```
if (Get-Module PSReadLine) { Set-PSReadLineOption -HistorySaveStyle SaveNothing }
$env:POSTGRES_PASSWORD = '<password-postgresql>'
$env:DJANGO_SECRET_KEY = '<stringa-casuale-almeno-32-caratteri>'
$env:LOCAL_API_SECRET = '<segreto-locale-almeno-12-caratteri>'
$env:REMOTE_API_SECRET = '<segreto-remoto-almeno-12-caratteri>'
$env:BRIDGE_API_SECRET = '<segreto-bridge-almeno-12-caratteri>'
```

Usare valori distinti e chiudere la sessione al termine. I segreti restano nell'ambiente dei processi e non entrano nell'archivio, nello stato installato, nei log o nella cronologia PowerShell persistente.

Per la prova offline, `REMOTE_API_SECRET` è un segreto della sorgente sintetica: non è il Bearer Altrivista. Il Bearer Altrivista serve soltanto alla migrazione remota estesa della sezione 7 ed è fornito separatamente o su richiesta. Non servono credenziali del pannello Altrivista né password del suo database.

5. Configurare e verificare

Adattare i quattro percorsi. Il comando crea `..\drive-aura-51-runtime`, un ambiente virtuale, un `CATALINA_BASE` isolato e il WAR corretto. Prepara due database distinti: il database operativo resta vuoto e pronto per i dati reali; il database di verifica riceve la fixture sintetica da 22 righe e 22 lotti.

```
.\installer\Configure-DriveAura.ps1 `
-PythonPath 'C:\Python312\python.exe' `
-JavaHome 'C:\Program Files\Java\jdk-17' `
-TomcatHome 'C:\apache-tomcat-11' `
-PostgresBin 'C:\Program Files\PostgreSQL\18\bin' `
-PostgresUser 'postgres' `
-PostgresDatabase 'drive_aura_51' `
-VerificationDatabase 'drive_aura_51_verify'
```

Il comando non modifica il Tomcat installato: copia solo la configurazione necessaria sotto `..\drive-aura-51-runtime\tomcat-base`. L'installazione Python usa sette wheel locali con `--no-index` e `--require-hashes`. Se `-VerificationDatabase` è omissso, il nome viene derivato aggiungendo `_verify` al database operativo.

I timeout PostgreSQL predefiniti sono: connessione 10 s (1-60), lock 10000 ms (100-120000), query 120000 ms e transazione inattiva 120000 ms (1000-600000). Le quattro variabili `POSTGRES_*_TIMEOUT_*` documentate nel README Django permettono di modificarli con soli interi negli intervalli.

L'esito corretto termina con:

```
PASS: verifica sintetica; righe=22; lotti=22; dataset=199452...418a7.
PASS: configurazione completata in ... secondi.
```

La prova pulita del 25 luglio 2026 ha usato Python 3.12.10 x64, Java 23.0.2, Tomcat 11.0.24 e PostgreSQL 18.4 con autenticazione SCRAM. Con rete resa indisponibile e pacchetto estratto in un percorso con spazi, configurazione e verifica sono terminate in 43,290 secondi misurati dal configuratore e 43,571 secondi wall-clock. La stessa prova con Tomcat 9.0.120 è terminata in 38,822 secondi. La verifica completa ha un watchdog predefinito di 180 secondi, configurabile con `-VerificationTimeoutSeconds` fra 60 e 240 secondi. In caso di errore o timeout mostra le code dei log, arresta soltanto l'albero di processi registrato e verifica che le porte siano state liberate.

6. Cosa verifica il comando

Il configuratore controlla versione e architettura dei runtime, compatibilità Java/Tomcat/WAR, PostgreSQL, hash di pacchetto-wheel-WAR-dump, installazione Django senza rete, migrazioni, salute, readiness, migrazione delle otto entità, digest, vincoli, idempotenza e arresto dei processi avviati dal controllo.

La verifica rapida usa dati sintetici leggibili. La migrazione massiva PHP/PDO di 36.176 righe è un collaudo storico T07 separato: non fa parte della verifica standard di consegna e richiede una sorgente remota conforme.

7. Servizio PHP remoto: sorgente reale opzionale

La consegna e la verifica standard sono **offline e locali**: completare le sezioni 1-6. Non è richiesto un account Altrivista né una migrazione reale per valutare il pacchetto.

L'endpoint Altrivista documentato in `ALTRIVISTA.md` è disponibile come sorgente reale opzionale. Il certificato HTTPS è valido; il manifest espone il dataset T07 da 36.176 righe con conteggi, digest e `datasetId` coincidenti. Una migrazione osservata attraverso Tomcat, Django e PostgreSQL ha completato 364 lotti ed è risultata idempotente; sono stati verificati entrambi i WAR.

Usare il remoto soltanto con rete disponibile e con il Bearer ricevuto fuori dal pacchetto. Non disabilitare TLS e non registrare il token. Sul server i segreti stanno in `remote-php/config/local.php`, fuori da `public`, mai nel pacchetto né in `.htaccess`. La

durata della migrazione massiva non fa parte del controllo rapido offline delle sezioni 1-6.

Per il test esteso, dopo aver configurato un PostgreSQL locale vuoto e avviato Django/Tomcat con `Start-DriveAura.ps1`, usare il comando già incluso e limitato nel tempo:

```
.\tools\verify-mass-migration.ps1 `
-RemoteBaseUrl 'https://motorizzami.altervista.org/drive-aura-api/remote-php/public' `
-LocalBaseUrl 'http://127.0.0.1:8000' `
-BridgeBaseUrl 'http://127.0.0.1:8080' `
-TimeoutSeconds 1800 -Repeat
```

È una migrazione estesa con rete: attende 36.176 righe, 364 lotti e il `datasetId` T07, quindi non è un controllo da cinque minuti. Non esiste un BAT separato per evitare di nascondere prerequisiti e credenziali della prova reale.

8. Arrestare

```
.\installer\Stop-DriveAura.ps1 `
-StatePath '..\drive-aura-51-runtime\install-state.json'
```

Attendere `PASS: processi Drive Aura arrestati`. Lo script usa il registro dei PID creato all'avvio e prova prima l'arresto Tomcat ordinato. Prima di forzare la chiusura verifica PID, percorso eseguibile e istante di avvio, quindi arresta soltanto i discendenti appartenenti a quell'albero.

9. Risolvere i problemi

Python non trovato o incompatibile

Sintomo: `Python 3.12 x64 compatibile non trovato`. Soluzione: installare Python 3.12 x64 completo; l'edizione embeddable non contiene `venv`.

Java o Tomcat incompatibile

Sintomo: richiesta Java 17, Tomcat non riconosciuto o Tomcat 10 rifiutato. Soluzione: abbinare Java 8+ a Tomcat 9 oppure Java 17+ a Tomcat 11 e passare la directory che contiene `bin\catalina.bat`. Tomcat 10 non è compatibile.

PostgreSQL non raggiungibile

Sintomo: errore di readiness o autenticazione. Soluzione: avviare PostgreSQL, controllare host e porta, verificare l'utente e reimpostare `POSTGRES_PASSWORD` nella sessione.

Porta occupata

Sintomo: porta Django, Tomcat, arresto o sorgente sintetica occupata. Soluzione: non terminare il processo segnalato; rilanciare il configuratore con porte libere, per esempio `-DjangoPort 18000 -TomcatPort 18080 -TomcatShutdownPort 18005 -SyntheticRemotePort 18081`.

Segreto mancante

Sintomo: `Segreto mancante` oppure HTTP 401. Soluzione: impostare nuovamente tutte le variabili della sezione 4 nella stessa finestra PowerShell.

Wheel, WAR, dump o checksum alterato

Sintomo: errore SHA-256 o integrità. Soluzione: non usare il file; eliminare la cartella estratta, verificare il checksum esterno dello ZIP e riestrarre il pacchetto originale.

Rete assente

Non è un errore per installazione e prova sintetica. Il configuratore non usa download. La rete serve soltanto quando si collega al PHP Altervista reale.

Percorso Windows troppo lungo

Sintomo: `Percorso Windows troppo lungo` oppure estrazione incompleta. Soluzione: non spostare la copia parziale; riestrarre lo ZIP originale direttamente in `C:\DriveAura51` e rilanciare il controllo di integrità.

Database, percorso o versione

Se il database di verifica non è vuoto, indicare un nome nuovo con `-VerificationDatabase`: il configuratore non cancella né sovrascrive dati. Se `python`, `java` o `psql` non sono riconosciuti, inserire nel BAT il percorso completo di `python.exe`, JDK, Tomcat e `bin` PostgreSQL, senza modificare il PATH. Per versioni incompatibili usare Python 3.12 x64, PostgreSQL 14-18 e Tomcat 9/11 con la Java indicata nella sezione 1.

10. Contenuto utile

`verifica-rapida.bat`, `installer`, `artifacts` (due WAR), `wheelhouse`, `database` (dump e checksum), `source` (PHP, Java, Django) e `pdf` (manuale e scelte progettuali).